

OPERATING SYSTEMS

DEADLOCK

Comprehensive Exam Notes

Topics Covered:

Deadlock Detection (Conditions) • Deadlock Avoidance • Deadlock Prevention
Deadlock Resolution Algorithm • Banker's Algorithm • Numericals & Practice Questions

CS-2006 | FAST NUCES | Spring 2026

TABLE OF CONTENTS

1. Introduction to Deadlock	3
2. Conditions for Deadlock.....	4
3. Resource Allocation Graphs (RAG)	6
4. Deadlock Prevention.....	8
5. Deadlock Avoidance	11
6. Banker's Algorithm.....	13
7. Deadlock Detection.....	19
8. Deadlock Recovery / Resolution	23
9. Comparison: Prevention vs Avoidance vs Detection	25
10. Exam Practice Questions – Theory & Reasoning.....	26
11. Exam Practice Questions – Numericals	30
12. Quick Revision Summary.....	36

1. Introduction to Deadlock

A deadlock is a situation in operating systems where a set of processes is permanently blocked, each waiting for a resource that is held by another process in the same set. None of the processes can proceed, release resources, or be awakened — the system is at a standstill.

1.1 Formal Definition

A set of processes is deadlocked when each process in the set is blocked, awaiting an event that can only be triggered by another blocked process in the same set.

□ Key Insight

Deadlock involves processes competing for the same set of resources. Once deadlock occurs, no efficient algorithmic solution exists to directly resolve it without external intervention (process termination or resource preemption).

1.2 Real-World Analogy

Imagine four cars at a four-way intersection, each blocking the next car from passing. Car A needs to pass where Car B is, Car B needs where Car C is, Car C needs where Car D is, and Car D needs where Car A is. None can move — this is deadlock!

1.3 Deadlock vs. Starvation

Aspect	Deadlock	Starvation
Definition	All processes permanently blocked	Process waits indefinitely but others proceed
Progress	No process makes progress	Some processes make progress
Cause	Circular resource dependency	Resource allocation priority/aging

2. Conditions for Deadlock

Deadlock can arise if and only if ALL FOUR of the following conditions hold simultaneously. These were first identified by E.W. Dijkstra and later refined by Coffman et al. (1971).

2.1 The Four Necessary Conditions

Condition 1: Mutual Exclusion

At least one resource must be held in a non-sharable mode. Only one process can use the resource at any given time. If another process requests that resource, it must wait until the resource is released.

- Example: A printer cannot be shared between two processes simultaneously.
- If resources were sharable (like read-only files), mutual exclusion would not apply.

Condition 2: Hold and Wait

A process must be holding at least one resource and waiting to acquire additional resources that are currently held by other processes.

- Example: Process P1 holds Resource R1 and is waiting for Resource R2 (which P2 holds).
- If processes had to request all resources at once before starting, this condition wouldn't arise.

Condition 3: No Preemption

Resources cannot be preempted (forcibly taken away). A resource can only be released voluntarily by the process holding it, after that process has completed its task.

- Example: A process holding a file lock cannot have that lock taken away forcibly.
- If the OS could forcibly reclaim resources, this condition would be eliminated.

Condition 4: Circular Wait

A set of waiting processes $\{P_0, P_1, \dots, P_n\}$ must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., and P_n is waiting for a resource held by P_0 .

- This forms a circular chain of dependencies — the most direct cause of deadlock.
- Example: P_1 waits for R_2 (held by P_2), P_2 waits for R_3 (held by P_3), P_3 waits for R_1 (held by P_1).

❑ Critical Exam Point

The first THREE conditions (Mutual Exclusion, Hold-and-Wait, No Preemption) are NECESSARY but NOT SUFFICIENT for deadlock. All FOUR conditions together are both necessary and sufficient. In other words: if circular wait exists along with the first three, deadlock WILL occur. Preventing any ONE condition prevents deadlock.

2.2 Conditions Summary Table

Condition	Description	Prevention Strategy
-----------	-------------	---------------------

Mutual Exclusion	Resource held exclusively	Make resources sharable (not always possible)
Hold and Wait	Hold some, wait for more	Request all resources upfront
No Preemption	Resources not forcibly removed	Allow OS to preempt resources
Circular Wait	Circular chain of waiting	Impose ordering on resource requests

3. Resource Allocation Graphs (RAG)

A Resource Allocation Graph (RAG) is a directed graph used to describe and analyze the state of resource allocation in a system. It visually represents which resources are allocated to which processes and what resources are being requested.

3.1 Graph Components

- Vertices (Nodes): Two types — Processes (circles: P1, P2, ...) and Resources (rectangles: R1, R2, ...)
- Resource instances: Dots inside resource rectangles represent the number of instances available.
- Assignment Edge (Resource → Process): Indicates the resource is currently allocated to the process.
- Request Edge (Process → Resource): Indicates the process is requesting the resource.

3.2 RAG Example (from slides)

Consider the following system state:

- R1: 1 instance, R2: 2 instances, R3: 1 instance, R4: 3 instances
- P1: Holds R2 (1 instance), Waiting for R1
- P2: Holds R1 (1 instance) and R2 (1 instance), Waiting for R3
- P3: Holds R3 (1 instance)

RAG Text Representation:

```
[P1] —request→ [R1] ←hold— [P2]
[P1] ←hold— [R2] ←hold— [P2]
[P2] —request→ [R3] ←hold— [P3]
```

Deadlock? YES — P1 waits for R1 (held by P2),
P2 waits for R3 (held by P3), P3 holds R3 but
needs nothing — P3 can finish, releasing R3 to P2,
P2 finishes, releasing R1 to P1. NO DEADLOCK here.

3.3 Deadlock Detection via RAG

- If the RAG contains NO cycle → No deadlock
- If the RAG contains a cycle:
 - Single instance per resource type: Cycle = DEADLOCK (necessary and sufficient)
 - Multiple instances per resource type: Cycle may or may not indicate deadlock (need further analysis)

3.4 Wait-For Graph (WFG)

A simplification of RAG used when each resource type has only ONE instance. Resource nodes are removed; an edge $P_i \rightarrow P_j$ means P_i is waiting for a resource currently held by P_j .

- Cycle in WFG = Deadlock (for single-instance resources)
- Detecting cycles requires $O(n^2)$ operations, where n = number of processes

4. Deadlock Prevention

Deadlock Prevention works by designing the system such that at least ONE of the four necessary conditions for deadlock CANNOT hold. This is a static approach applied at system design time.

□ Strategy

Design a system in such a way that the possibility of deadlock is permanently excluded. Two main methods: (1) Indirect — prevent one of the three individual conditions, (2) Direct — prevent circular wait.

4.1 Preventing Mutual Exclusion

Some resources are inherently non-sharable (like printers, tape drives). Mutual exclusion cannot be denied for non-sharable resources. However, for sharable resources (e.g., read-only files), multiple processes can access simultaneously.

- Strategy: Make resources sharable wherever possible.
- Limitation: Cannot be applied to all resources — inherently non-sharable ones (like printers) require mutual exclusion.
- Verdict: Generally NOT practical as a deadlock prevention method.

4.2 Preventing Hold and Wait

To prevent Hold and Wait, we must guarantee that when a process requests a resource, it does not hold any other resources.

Protocol 1: Request All Resources at Once

- Each process must request and be allocated ALL necessary resources before it begins execution.
- Advantage: Simple to implement.
- Disadvantage: Low resource utilization — resources may be held but not used for a long time; starvation is possible.

Protocol 2: Release Before Requesting New Resources

- Allow a process to request resources only when it has none.
- If a process needs additional resources, it must release all currently held resources first, then request all needed resources together.

4.3 Preventing No Preemption

Allow resources to be preempted (forcibly taken away) from processes.

Protocol:

- If a process holding resources requests a new resource that cannot be immediately allocated, all resources currently being held are preempted (released).
- The preempted resources are added to the list of resources the process is waiting for.

- The process is restarted only when it can get all its old and new resources.
- Applies well to: Resources whose state can be easily saved and restored (CPU registers, memory).
- Does NOT apply well to: Printers, tape drives (state cannot be easily saved mid-use).

4.4 Preventing Circular Wait (Most Practical Method)

Impose a total ordering of all resource types and require that each process requests resources in increasing order of enumeration.

Algorithm:

- Assign a unique integer priority (ordering) to each resource type: $R_1=1$, $R_2=2$, $R_3=3$, etc.
- A process can only request resource R_j if it currently holds no resource R_i where $F(R_i) \geq F(R_j)$, where F is the ordering function.
- If a process needs R_i and holds R_j where $F(R_i) < F(R_j)$, it must release R_j first.

Example

Resources ordered: Tape Drive (1) < Disk Drive (2) < Printer (3) Process P must request Tape Drive before Disk Drive before Printer. This eliminates the possibility of circular wait because the cycle requires at least one process to request a resource in decreasing order.

- Advantage: Most practical prevention method; widely used.
- Disadvantage: May force unnecessary waits; ordering may not reflect actual usage patterns.

4.5 Summary: Deadlock Prevention Methods

Condition	Prevention Method	Advantage	Disadvantage
Mutual Exclusion	Make sharable	Simple	Not always feasible
Hold & Wait	Request all at once / Release before request	Deadlock-free	Low utilization, starvation
No Preemption	Allow preemption	Resource recovery	Not for all resources
Circular Wait	Impose resource ordering	Most practical	May cause unnecessary waits

5. Deadlock Avoidance

Deadlock avoidance is a dynamic approach where the OS makes real-time decisions about resource allocation to ensure the system never enters an unsafe state. Unlike prevention, it requires advance information about resource needs.

□ Key Requirement

Each process must declare in advance the MAXIMUM number of resources of each type it may ever need. The OS uses this information to decide whether granting a request will keep the system in a safe state.

5.1 Safe State vs. Unsafe State

Safe State

A system is in a SAFE STATE if there exists at least one safe sequence of processes such that, for each process P_i in the sequence, the resources that P_i can still request can be satisfied by the currently available resources plus the resources held by all P_j where $j < i$.

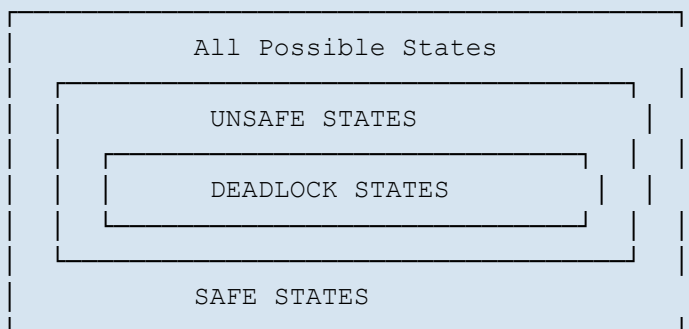
- In other words: Each process can eventually get all the resources it needs, complete, and release them for the next process.
- Safe state $\Rightarrow$ No deadlock possible (currently or in the future through this sequence).

Unsafe State

A state that is NOT safe. An unsafe state does NOT necessarily mean deadlock has occurred — it means deadlock is POSSIBLE if processes make requests in a worst-case manner.

- Unsafe state $\Rightarrow$ Possible deadlock (but not guaranteed).

State Hierarchy Diagram:



Avoidance Goal: Stay ONLY in Safe States

5.2 Two Approaches to Deadlock Avoidance

Approach 1: Process Initiation Denial

A new process P_{new} is only started if the maximum claims of ALL current processes PLUS those of P_{new} can be simultaneously satisfied by the total available resources.

- Condition: $\text{Resources_total} \geq \text{Max}(\text{all current processes}) + \text{Max}(P_{\text{new}})$
- Very conservative — assumes the worst case (all processes will request maximum simultaneously).
- Often leads to rejecting new processes unnecessarily.

Approach 2: Resource Allocation Denial (Banker's Algorithm)

This is the primary and widely-used approach. Before granting any resource request, the system checks whether granting it would leave the system in a safe state. If yes, grant; if no, make the process wait.

6. Banker's Algorithm

The Banker's Algorithm, developed by Edsger Dijkstra (1965), is a resource allocation and deadlock avoidance algorithm. The name comes from a bank that checks whether granting a loan will leave enough funds to satisfy all other customers' needs.

6.1 Preconditions

- Fixed number of processes and resource types in the system.
- Each process must declare its MAXIMUM resource requirements before execution.
- Processes may wait if resources are unavailable.
- Processes must return all resources in a finite amount of time.
- Multiple instances of each resource type may exist.

6.2 Data Structures

Let n = number of processes, m = number of resource types.

Variable	Type	Description
Available	Vector $[m]$	$Available[j] = k$ means k instances of R_j available
Max	Matrix $[n \times m]$	$Max[i][j] = k$ means P_i may request up to k instances of R_j
Allocation	Matrix $[n \times m]$	$Allocation[i][j] = k$ means P_i is currently allocated k instances of R_j
Need	Matrix $[n \times m]$	$Need[i][j] = Max[i][j] - Allocation[i][j]$ (remaining need)

Formula

$Need[i][j] = Max[i][j] - Allocation[i][j]$ This tells us how many MORE resources process P_i may still request for resource type R_j .

6.3 Safety Algorithm

This algorithm checks whether the current system state is SAFE. It finds a safe sequence if one exists.

SAFETY ALGORITHM (Step-by-Step):

Step 1: Initialize:

```
Work[m] = Available[m]
Finish[n] = {false, false, ..., false}
```

Step 2: Find index i such that BOTH conditions hold:

```
(a) Finish[i] == false
```

```
(b) Need[i] <= Work    (element-wise comparison)
If no such i exists → go to Step 4
```

```
Step 3: Work = Work + Allocation[i]
Finish[i] = true
Go to Step 2
```

```
Step 4: If Finish[i] == true for ALL i:
        → System is in SAFE STATE
Else:
        → System is in UNSAFE STATE (potential deadlock)
```

6.4 Resource-Request Algorithm

When process P_i makes a request $Request_i$, this algorithm decides whether to grant it immediately.

RESOURCE-REQUEST ALGORITHM:

```
Step 1: If Request_i <= Need_i → Go to Step 2
        Else: ERROR – process exceeded maximum claim
```

```
Step 2: If Request_i <= Available → Go to Step 3
        Else: Pi must WAIT (resources not available)
```

```
Step 3: PRETEND to allocate (tentatively):
        Available      = Available - Request_i
        Allocation_i    = Allocation_i + Request_i
        Need_i         = Need_i - Request_i
```

```
Step 4: Run Safety Algorithm on the new state:
        If SAFE → Permanently allocate (grant request)
        If UNSAFE → Rollback changes, Pi must WAIT
```

6.5 Worked Example: Banker's Algorithm

This is the standard exam-style example. Study every step carefully.

Given Data (5 processes, 3 resource types: A, B, C)

Total Resources: A=10, B=5, C=7

	Alloc A	Alloc B	Alloc C	Max A	Max B	Max C	Need A	Need B	Need C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0

P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

	A	B	C	Notes
Total Instances	10	5	7	Given in problem
Total Allocated	7	2	5	Sum of Allocation column
Available	3	3	2	Total – Allocated

Step-by-Step Safety Algorithm Execution

Initial: Work = [3,3,2], Finish = [F,F,F,F,F]

Iteration 1:

Check P0: Need=[7,4,3] <= Work=[3,3,2]? NO (7>3). Skip.
 Check P1: Need=[1,2,2] <= Work=[3,3,2]? YES (1<=3, 2<=3, 2<=2)
 → Grant P1: Work = [3,3,2]+[2,0,0] = [5,3,2], Finish[1]=true

Iteration 2:

Check P0: Need=[7,4,3] <= Work=[5,3,2]? NO. Skip.
 Check P2: Need=[6,0,0] <= Work=[5,3,2]? NO (6>5). Skip.
 Check P3: Need=[0,1,1] <= Work=[5,3,2]? YES
 → Grant P3: Work = [5,3,2]+[2,1,1] = [7,4,3], Finish[3]=true

Iteration 3:

Check P0: Need=[7,4,3] <= Work=[7,4,3]? YES
 → Grant P0: Work = [7,4,3]+[0,1,0] = [7,5,3], Finish[0]=true

Iteration 4:

Check P2: Need=[6,0,0] <= Work=[7,5,3]? YES
 → Grant P2: Work = [7,5,3]+[3,0,2] = [10,5,5], Finish[2]=true

Iteration 5:

Check P4: Need=[4,3,1] <= Work=[10,5,5]? YES
 → Grant P4: Work = [10,5,5]+[0,0,2] = [10,5,7], Finish[4]=true

All Finish = TRUE → SAFE STATE

Safe Sequence: <P1, P3, P0, P2, P4>

Resource-Request Example: P1 requests (1,0,2)

Request P1 = (1,0,2)

Step 1: Request[1]=(1,0,2) <= Need[1]=(1,2,2)? YES (1<=1, 0<=2, 2<=2). OK

Step 2: Request[1]=(1,0,2) <= Available=(3,3,2)? YES (1<=3, 0<=3, 2<=2). OK

Step 3: Pretend to allocate:

Available = $[3,3,2] - [1,0,2] = [2,3,0]$

Allocation[1] = $[2,0,0] + [1,0,2] = [3,0,2]$

Need[1] = $[1,2,2] - [1,0,2] = [0,2,0]$

Step 4: Run Safety Algorithm on new state...

Safe Sequence found: $\langle P1, P3, P4, P0, P2 \rangle$

→ SAFE → GRANT the request to P1

7. Deadlock Detection

In deadlock detection, the OS does NOT try to prevent or avoid deadlock. Instead, it periodically runs a detection algorithm to determine whether a deadlock has occurred, and then applies a recovery scheme.

7.1 Single Instance Resources — Wait-For Graph

- Maintain a wait-for graph derived from the RAG (remove resource nodes).
- $P_i \rightarrow P_j$ means P_i is waiting for a resource currently held by P_j .
- Periodically run a cycle-detection algorithm: if cycle exists $\rightarrow$ DEADLOCK.
- Cycle detection in a directed graph requires $O(n^2)$ operations where n = number of processes.

7.2 Multiple Instance Resources — Detection Algorithm

This algorithm is similar to the Banker's Safety Algorithm but applied to detect (not avoid) deadlock. It uses ACTUAL requests, not maximum needs.

DEADLOCK DETECTION ALGORITHM:

Step 1: Initialize:

Work[m] = Available[m]

For each i : if Allocation[i] == 0 $\rightarrow$ Finish[i] = true
else $\rightarrow$ Finish[i] = false

Step 2: Find index i such that BOTH:

(a) Finish[i] == false

(b) Request[i] <= Work (Request = ACTUAL current request)

If no such $i \rightarrow$ go to Step 4

Step 3: Work = Work + Allocation[i]

Finish[i] = true

Go to Step 2

Step 4: If Finish[i] == FALSE for some i :

$\rightarrow$ DEADLOCK exists! P_i is deadlocked.

If all Finish[i] == true:

$\rightarrow$ No deadlock

Key Difference: Safety vs Detection Algorithm

Safety Algorithm (Avoidance): Uses NEED matrix (Max – Allocation) — pessimistic view of future. Detection Algorithm: Uses REQUEST matrix (actual current request) — optimistic view; assumes processes not requesting will complete and release resources.

7.3 Worked Example: Detection Algorithm

From lecture slides — 5 processes (P0–P4), 3 resource types A(7), B(2), C(6):

	Alloc A	Alloc B	Alloc C	Req A	Req B	Req C	Avail A	Avail B	Avail C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Available at T0 = [0, 0, 0]

Execution:

Initial: Work=[0,0,0]

Finish: P0=false, P1=false, P2=false, P3=false, P4=false

(Note: All have non-zero Allocation, so all start as false)

Check P0: Request[0]=[0,0,0] <= Work=[0,0,0]? YES

Work = [0,0,0]+[0,1,0] = [0,1,0], Finish[0]=true

Check P1: Request[1]=[2,0,2] <= Work=[0,1,0]? NO (2>0). Skip

Check P2: Request[2]=[0,0,0] <= Work=[0,1,0]? YES

Work = [0,1,0]+[3,0,3] = [3,1,3], Finish[2]=true

Check P3: Request[3]=[1,0,0] <= Work=[3,1,3]? YES

Work = [3,1,3]+[2,1,1] = [5,2,4], Finish[3]=true

Check P1: Request[1]=[2,0,2] <= Work=[5,2,4]? YES

Work = [5,2,4]+[2,0,0] = [7,2,4], Finish[1]=true

Check P4: Request[4]=[0,0,2] <= Work=[7,2,4]? YES

Work = [7,2,4]+[0,0,2] = [7,2,6], Finish[4]=true

All Finish=TRUE → NO DEADLOCK

Safe Sequence: <P0, P2, P3, P1, P4>

7.4 When to Run Detection?

- Trade-off: Frequent detection = more overhead; infrequent = longer deadlock duration.
- Option 1: Run every time a resource request cannot be granted immediately.
- Option 2: Run at fixed time intervals (e.g., every hour).
- Option 3: Run when CPU utilization drops below a threshold (could indicate deadlock).

8. Deadlock Recovery / Resolution Algorithm

Once a deadlock is detected, the system must recover from it. There are two main strategies: Process Termination and Resource Preemption.

8.1 Strategy 1: Process Termination

Option A: Abort ALL Deadlocked Processes

- Most drastic approach — all deadlocked processes are killed.
- Advantage: Guaranteed to break the deadlock.
- Disadvantage: Very expensive — all partial computations lost; processes must restart from scratch.

Option B: Abort One Process at a Time

- Abort processes one by one, running the detection algorithm after each abort.
- Continue until deadlock is broken.
- Less disruptive, but requires running detection algorithm repeatedly (higher overhead).

8.2 Choosing Which Process to Terminate

When aborting one process at a time, the system must decide which process to abort next. Factors considered (to minimize cost):

- Priority of the process (lower priority = more likely to abort)
- How long the process has been running and how close it is to completion
- Number and types of resources the process has used
- Resources the process needs to complete
- Number of other processes that would need to be terminated
- Whether the process is interactive or batch (interactive = harder to terminate)

8.3 Strategy 2: Resource Preemption

Forcibly take resources away from some processes and give them to others to break the deadlock cycle. Involves three key issues:

Issue 1: Selecting a Victim

- Choose which process and which resources to preempt.
- Minimize cost: choose victim with least partial work, lowest priority, or fewest resources needed.

Issue 2: Rollback

- After preempting resources from a process, the process cannot continue from where it left off.
- Must roll back to a SAFE STATE (a checkpoint saved earlier).
- Options: Total Rollback (restart from scratch) or Partial Rollback (restart from last checkpoint).

Issue 3: Starvation

- If the same process is always chosen as victim, it may never complete — STARVATION.
- Solution: Include the number of rollbacks in the cost factor — process cannot be victimized infinitely.

8.4 Recovery Strategy Comparison

Strategy	Method	Advantage	Disadvantage
Termination	Abort all deadlocked	Guaranteed deadlock break	Expensive; all work lost
Termination	Abort one at a time	Less disruptive	Overhead of repeated detection
Preemption	Take resources, rollback	Work may be partially saved	Starvation; complex rollback

9. Comparison: Prevention vs Avoidance vs Detection

Factor	Prevention	Avoidance	Detection
When applied	At design/compile time	At runtime	At runtime (after deadlock)
Knowledge needed	None (structural rules)	Max resource needs	None in advance
Approach	Eliminate one condition	Avoid unsafe states	Allow deadlock, then fix
Resource Utilization	Low	Medium	High
Throughput	Low	Medium	High (until deadlock)
Overhead	Low	High (safety check each time)	Medium (periodic detection)
Main Algorithm	Ordering, preemption rules	Banker's Algorithm	Wait-for graph / Detection Algo

10. Exam Practice Questions – Theory & Reasoning

These questions test conceptual understanding, analysis, and ability to reason about deadlock scenarios.

Section A: Short Answer Questions

Q1. Define deadlock and explain what it means for a set of processes to be 'deadlocked'.

Model Answer

A deadlock occurs when a set of processes is permanently blocked because each process is waiting for an event that can only be triggered by another blocked process in the same set. No process in the set can ever proceed, release resources, or be awakened without external intervention. Deadlock typically involves processes competing for the same set of resources where circular dependencies have formed.

Q2. State and explain ALL four Coffman conditions for deadlock. Why are all four required?

Model Answer

1) Mutual Exclusion: Resources held exclusively by one process at a time. 2) Hold and Wait: A process holds resources while waiting for more. 3) No Preemption: Resources can only be voluntarily released. 4) Circular Wait: A circular chain of processes each waiting for a resource held by the next. All four are required because: The first three conditions enable the possibility of deadlock but do not guarantee it. Deadlock only ACTUALLY occurs when all four hold simultaneously. If ANY one condition is absent, the circular dependency that defines deadlock cannot form. This is why preventing any single condition prevents deadlock.

Q3. Distinguish between deadlock prevention and deadlock avoidance. Give one advantage and one disadvantage of each.

Model Answer

Prevention: Eliminates deadlock by designing the system so at least one of the four conditions can never hold. Applied statically at design time. Advantage: Simple, no runtime overhead. Disadvantage: Often overly restrictive, leads to poor resource utilization. Avoidance: Dynamically checks at runtime whether granting a request leads to an unsafe state; if unsafe, the request is denied. Advantage: Better resource utilization than prevention. Disadvantage: Requires advance knowledge of maximum resource needs; overhead of safety check on every request.

Q4. What is a 'safe state'? Why does being in a safe state guarantee freedom from deadlock?

Model Answer

A safe state is one in which there exists at least one safe sequence of all processes such that each process can obtain its remaining resource needs using the currently available resources plus resources released by processes that complete before it. A safe state guarantees no deadlock because the safe sequence shows a valid order in which all processes can complete without being permanently blocked. Even in the worst case (each process requests its maximum), the system can always find a way to satisfy all needs.

Q5. Why is deadlock detection preferred in some real systems over prevention or avoidance?

Model Answer

Deadlock detection allows the highest resource utilization because it places no restrictions on how resources are requested. In systems with many resources and processes where deadlock is rare, the overhead of constantly running prevention/avoidance is wasteful. Detection only incurs cost when it runs, and recovery only happens when actually needed. However, the cost is that deadlock may persist for some time before it is detected, and recovery may require aborting processes or losing work.

Section B: Analysis and Reasoning Questions

Q6. Explain why eliminating the 'circular wait' condition is considered the most practical deadlock prevention method.

Model Answer

Circular wait is targeted by imposing a total ordering on all resource types. Processes must request resources in strictly increasing order of their assigned numbers. This is practical because: (1) It doesn't restrict how many resources a process can hold simultaneously, (2) It doesn't require upfront knowledge of all needs, (3) The ordering can often be defined to match natural usage patterns, (4) Implementation is straightforward — processes just follow the ordering rule. Other methods have more severe drawbacks: mutual exclusion cannot be removed for non-sharable resources, hold-and-wait elimination requires all-or-nothing requests (wasteful), and preemption doesn't work for all resource types.

Q7. A system has the following RAG: P1 holds R1, requests R2; P2 holds R2, requests R3; P3 holds R3, requests R1. Is there a deadlock? Explain using the four conditions.

Model Answer

YES, there is a deadlock. Verify all four conditions: 1) Mutual Exclusion: Each resource (R1, R2, R3) is held exclusively by one process. ✓ 2) Hold and Wait: P1 holds R1 and waits for R2; P2 holds R2 and waits for R3; P3 holds R3 and waits for R1. ✓ 3) No Preemption: No resource is taken forcibly. ✓ 4) Circular Wait: P1→waits for R2 (held by P2)→P2 waits for

R3 (held by P3)→P3 waits for R1 (held by P1)→ forms a cycle. ✓ All four conditions hold, so deadlock exists. The safe sequence does not exist for these three processes.

Q8. Explain the concept of rollback in deadlock recovery. What is starvation in this context and how is it prevented?

Model Answer

Rollback: When a process is selected as a victim in resource preemption, its resources are taken away. Since it cannot continue from its current state (resources are missing), it must roll back to a previously safe checkpoint state. Total rollback means restarting from scratch; partial rollback means returning to the last saved checkpoint. Starvation: If the system always selects the same process as the victim (e.g., the one with the lowest priority or fewest resources), that process may never complete — it gets rolled back every time it makes progress. This is starvation. Prevention: Include the number of rollbacks in the cost factor used for victim selection. As a process is rolled back more times, its cost increases, making other processes more likely to be chosen as victims instead.

11. Exam Practice Questions – Numericals

Work through these step-by-step. Show all work for full marks.

Numerical 1: Safety Algorithm

QUESTION:

Consider a system with 5 processes (P0–P4) and 3 resource types (A, B, C) with instances (10, 5, 7). At time T0:

Process	Allocation (A,B,C)	Max (A,B,C)
P0	0 1 0	7 5 3
P1	2 0 0	3 2 2
P2	3 0 2	9 0 2
P3	2 1 1	2 2 2
P4	0 0 2	4 3 3

(a) Compute the Need matrix and Available vector.

(b) Using the Safety Algorithm, determine if the system is in a safe state. If safe, find the safe sequence.

SOLUTION:

(a) Need = Max - Allocation:

$$P0: [7, 5, 3] - [0, 1, 0] = [7, 4, 3]$$

$$P1: [3, 2, 2] - [2, 0, 0] = [1, 2, 2]$$

$$P2: [9, 0, 2] - [3, 0, 2] = [6, 0, 0]$$

$$P3: [2, 2, 2] - [2, 1, 1] = [0, 1, 1]$$

$$P4: [4, 3, 3] - [0, 0, 2] = [4, 3, 1]$$

$$\text{Total Allocated: } A=0+2+3+2+0=7, \quad B=1+0+0+1+0=2, \quad C=0+0+2+1+2=5$$

$$\text{Available} = [10, 5, 7] - [7, 2, 5] = [3, 3, 2]$$

(b) Safety Algorithm: Work=[3,3,2], Finish=[F,F,F,F,F]

Round 1: P1 Need=[1,2,2] <= [3,3,2]? YES

$$\text{Work}=[3, 3, 2] + [2, 0, 0] = [5, 3, 2], \quad \text{Finish}[1]=T$$

Round 2: P3 Need=[0,1,1] <= [5,3,2]? YES

$$\text{Work}=[5, 3, 2] + [2, 1, 1] = [7, 4, 3], \quad \text{Finish}[3]=T$$

Round 3: P0 Need=[7,4,3] <= [7,4,3]? YES

$$\text{Work}=[7, 4, 3] + [0, 1, 0] = [7, 5, 3], \quad \text{Finish}[0]=T$$

Round 4: P2 Need=[6,0,0] <= [7,5,3]? YES

$$\text{Work}=[7, 5, 3] + [3, 0, 2] = [10, 5, 5], \quad \text{Finish}[2]=T$$

Round 5: P4 Need=[4,3,1] <= [10,5,5]? YES

$$\text{Work}=[10, 5, 5] + [0, 0, 2] = [10, 5, 7], \quad \text{Finish}[4]=T$$

All Finish = TRUE → SAFE STATE
Safe Sequence: <P1, P3, P0, P2, P4>

Numerical 2: Resource-Request Algorithm

QUESTION (using Numerical 1 data):

Process P4 makes the request Request4 = (3, 3, 0). Using the Banker's Resource-Request Algorithm, determine if this request should be granted.

SOLUTION:

Request4 = (3, 3, 0)

Step 1: Request4=(3,3,0) ≤ Need4=(4,3,1)?
3≤4 ✓, 3≤3 ✓, 0≤1 ✓ → YES. Proceed.

Step 2: Request4=(3,3,0) ≤ Available=(3,3,2)?
3≤3 ✓, 3≤3 ✓, 0≤2 ✓ → YES. Proceed.

Step 3: Tentatively allocate:

Available = [3,3,2] - [3,3,0] = [0,0,2]
Allocation4 = [0,0,2] + [3,3,0] = [3,3,2]
Need4 = [4,3,1] - [3,3,0] = [1,0,1]

Step 4: Run Safety on new state (Work=[0,0,2]):

P0 Need=[7,4,3] ≤ [0,0,2]? NO
P1 Need=[1,2,2] ≤ [0,0,2]? NO
P2 Need=[6,0,0] ≤ [0,0,2]? NO
P3 Need=[0,1,1] ≤ [0,0,2]? NO (1>0 for B)
P4 Need=[1,0,1] ≤ [0,0,2]? NO (1>0 for A)

No process can proceed! UNSAFE STATE.

→ DENY the request. Rollback changes. P4 must WAIT.

Numerical 3: Deadlock Detection

QUESTION:

A system has 4 processes and 3 resource types (A: 4 instances, B: 2 instances, C: 3 instances). Snapshot:

Process	Allocation (A,B,C)	Request (A,B,C)
P0	0 1 0	0 0 0
P1	2 0 1	2 0 0
P2	0 0 1	0 0 2

P3 1 1 0 1 0 0

Available: A=1, B=0, C=2

Apply the Deadlock Detection Algorithm. Is the system deadlocked? If so, which processes are involved?

SOLUTION:

Initialize:

Work = [1,0,2]

Finish: P0=false (Alloc≠0), P1=false, P2=false, P3=false

Iteration 1:

P0: Request=[0,0,0] <= Work=[1,0,2]? YES

Work=[1,0,2]+[0,1,0]=[1,1,2], Finish[0]=true

Iteration 2:

P1: Request=[2,0,0] <= Work=[1,1,2]? NO (2>1)

P2: Request=[0,0,2] <= Work=[1,1,2]? YES

Work=[1,1,2]+[0,0,1]=[1,1,3], Finish[2]=true

Iteration 3:

P1: Request=[2,0,0] <= Work=[1,1,3]? NO (2>1)

P3: Request=[1,0,0] <= Work=[1,1,3]? YES

Work=[1,1,3]+[1,1,0]=[2,2,3], Finish[3]=true

Iteration 4:

P1: Request=[2,0,0] <= Work=[2,2,3]? YES

Work=[2,2,3]+[2,0,1]=[4,2,4], Finish[1]=true

All Finish=TRUE → NO DEADLOCK

Sequence: <P0, P2, P3, P1>

Numerical 4: Deadlock Detection with Deadlock Present

QUESTION (from lecture slides):

5 processes (P0–P4), 3 resource types A(7 inst), B(2 inst), C(6 inst). After P2 requests additional C:

Process	Allocation (A,B,C)	Request (A,B,C)
---------	--------------------	-----------------

P0	0 1 0	0 0 0
----	-------	-------

P1	2 0 0	2 0 2
----	-------	-------

P2	3 0 3	0 0 1
----	-------	-------

P3	2 1 1	1 0 0
----	-------	-------

P4	0 0 2	0 0 2
----	-------	-------

Available = [0, 0, 0]

Is there a deadlock? Which processes are deadlocked?

SOLUTION:

Work=[0,0,0], Finish=[F,F,F,F,F]
(P0 Alloc=[0,1,0]≠0 → Finish[0]=false)

Round 1:

P0: Request=[0,0,0] ≤ [0,0,0]? YES
Work=[0,0,0]+[0,1,0]=[0,1,0], Finish[0]=true

Round 2:

P1: Request=[2,0,2] ≤ [0,1,0]? NO
P2: Request=[0,0,1] ≤ [0,1,0]? NO (1>0 for C)
P3: Request=[1,0,0] ≤ [0,1,0]? NO (1>0 for A)
P4: Request=[0,0,2] ≤ [0,1,0]? NO (2>0 for C)
No process can proceed. Stop.

Finish: P0=true, P1=false, P2=false, P3=false, P4=false

→ DEADLOCK EXISTS

Deadlocked processes: P1, P2, P3, P4

P0 completed; the rest are permanently blocked in a cycle.

Numerical 5: Need Matrix Calculation + Safety Check

QUESTION:

A system has 3 processes and 3 resource types (A: 9, B: 3, C: 6 total instances). Allocation and Max are given below. Available = (3, 1, 1).

Process	Allocation (A,B,C)	Max (A,B,C)
P0	2 1 2	4 2 3
P1	3 1 2	9 1 5
P2	1 0 1	7 1 3

(a) Compute the Need matrix. (b) Is system in safe state? Find the safe sequence or show it is unsafe.

SOLUTION:

(a) Need = Max - Allocation:

P0: [4,2,3]-[2,1,2] = [2,1,1]
P1: [9,1,5]-[3,1,2] = [6,0,3]
P2: [7,1,3]-[1,0,1] = [6,1,2]

(b) Work=[3,1,1], Finish=[F,F,F]

Round 1: P0 Need=[2,1,1] ≤ [3,1,1]? YES
Work=[3,1,1]+[2,1,2]=[5,2,3], Finish[0]=true

Round 2: P1 Need=[6,0,3] ≤ [5,2,3]? NO (6>5 for A)
P2 Need=[6,1,2] ≤ [5,2,3]? NO (6>5 for A)

No process can proceed!

Finish: P0=true, P1=false, P2=false
→ UNSAFE STATE (potential deadlock)
Processes P1 and P2 cannot complete.

12. Quick Revision Summary

12.1 Key Formulas

Formula	Meaning
$\text{Need} = \text{Max} - \text{Allocation}$	Remaining resources process may still request
$\text{Available} = \text{Total} - \sum \text{Allocation}$	Currently free resources
$\text{Work} = \text{Available (init)}$	Working copy of available resources in algorithm
$\text{Work} += \text{Allocation}[i]$	When process i completes, it releases its resources

12.2 Algorithm Quick Reference

Algorithm	Uses Need or Request?	Purpose	Result
Safety	Need (Max-Alloc)	Check if state is safe	Safe/Unsafe + sequence
Resource-Request	Need + Available	Decide grant/deny	Grant or Wait
Detection	Request (actual)	Find deadlocked processes	Deadlocked set or None

12.3 Common Exam Mistakes to Avoid

- Confusing Need matrix (used in Banker's avoidance) with Request vector (used in detection).
 $\text{Need} = \text{Max} - \text{Allocation}$; Request = actual current request.
- Forgetting to check BOTH conditions in Step 2 of Safety/Detection: $\text{Finish}[i] == \text{false}$ AND $\text{Need/Request} \leq \text{Work}$.
- Not updating Work when a process completes: $\text{Work} = \text{Work} + \text{Allocation}[i]$.
- Confusing safe state with deadlock-free: safe state guarantees no deadlock in any future scenario; unsafe just means deadlock is POSSIBLE.
- Forgetting to rollback in Resource-Request algorithm if safety check fails — the tentative allocation must be reversed.
- Using total resources minus allocated to find Available — always verify: $\sum \text{Allocation} + \text{Available} = \text{Total}$.

□ Final Exam Tips

1. Always show your Need matrix calculation — it's worth marks. 2. For safety algorithm, track Work and Finish at each iteration in a table format. 3. State your conclusion explicitly: 'System is in a SAFE STATE. Safe sequence is <...>' 4. For resource-request: show all 3 steps before running safety algorithm. 5. For theory questions: always mention all 4 Coffman conditions with examples. 6. Remember: Safe $\Rightarrow$ No deadlock; Unsafe $\Rightarrow$ Possible (not guaranteed) deadlock.

— End of Notes —

CS-2006 Operating Systems | FAST NUCES Spring 2026